Using the framework in a regulated environment

Banks, insurers, pharma companies and pharmacovigilance teams all work under rules about how software is tested and how that testing is proved. This folder explains what the framework gives you for that, what it doesn't, and which documents to start from.

It helps you do the engineering well and produce evidence. It does not make a system "compliant" by itself. Compliance depends on your client's quality system, the people who review and approve the work, and the controls around this repository.

What you get

Need	What the framework provides
Traceability	Tests are linked to requirement IDs. `npm run traceability` builds a requirement-to-test matrix (CSV, Markdown, HTML).
Repeatable runs	Pinned application images, a lock file, environment profiles and a pipeline that starts from a clean state every time.
Evidence of a run	`npm run evidence` gathers the matrix, raw results, tool versions and image digests, with SHA-256 checksums of every file.
Controlled test data	Tests create their own data through the API and remove it. Nothing depends on real records. Builders produce synthetic data.
Read-only verification	The database layer reads what was stored (an order, a hashed password) and never writes.
Secrets handling	Credentials come from environment variables or the CI secret store, never from the code.
Visible known defects	A bug found in the system is written as a `@known-issue` test, so it is reported and tracked instead of ignored.
Change discipline	Pull request template, lint and type checks in CI, code owners for the files that define "validated".

What you still have to do

- **Validation documents.** A validation plan, risk assessment and summary report, signed by the right roles. Templates are in this folder, but your client's own templates take precedence.
- **Requirements.** The list in requirements.json is a sample for the demo application. For real work it comes from the client's specification, with their IDs.
- **Tool qualification.** Playwright, Node and the other tools are open-source software. Document what you use and why it is fit for purpose, in proportion to the risk. The manifest records the

exact versions.

- **Long-term storage.** CI artifacts expire. Store the signed-off evidence pack in the client's document system or in immutable storage. See repository-controls.md.
- **Access and approvals.** Branch protection, required reviews and a named approver for the final report. See repository-controls.md.
- **Where it runs.** Some clients don't allow public CI runners. See air-gapped-and-self-hosted.md.

Standard and regulated reports

Two kinds of report come out of the same tests, in separate folders so they are never confused:

	`reports/standard/`	`reports/regulated/`
For	developers and everyday runs	reviewers, auditors, clients
Content	Allure report and Playwright report	one validation report, traceability matrix, Allure report, every recording, manifest and checksums
Built by	`npm test`, then `npm run report:allure`	`npm run test:capture`, then `npm run report:validation`
Overwritten	by the next run	by the next validation run, so copy an approved pack elsewhere
Integrity	none	a checksum for every file

Run `npm run clean` before a validation run if the previous run was a normal one: `npm run test:capture` clears the raw results itself, but a normal `npm test` leaves earlier results in `reports/_raw`.

How a run becomes evidence

1. Every test names the requirement it verifies:

   ```
   test('a customer can sign in', { tag: [Tags.smoke, ...requirement('REQ-AUTH-001')] },
   ...)
   ```

   Put the tag on a `test.describe` to cover every test inside it. A test can list several IDs.

2. The ID is an ordinary tag, so the Allure report's Tags filter lists it, and `npx playwright test --grep @REQ-ORD-001` runs the tests for one requirement.

3. `npm run report:validation` reads the results of the last run and writes the matrix to `reports/regulated/traceability/`:

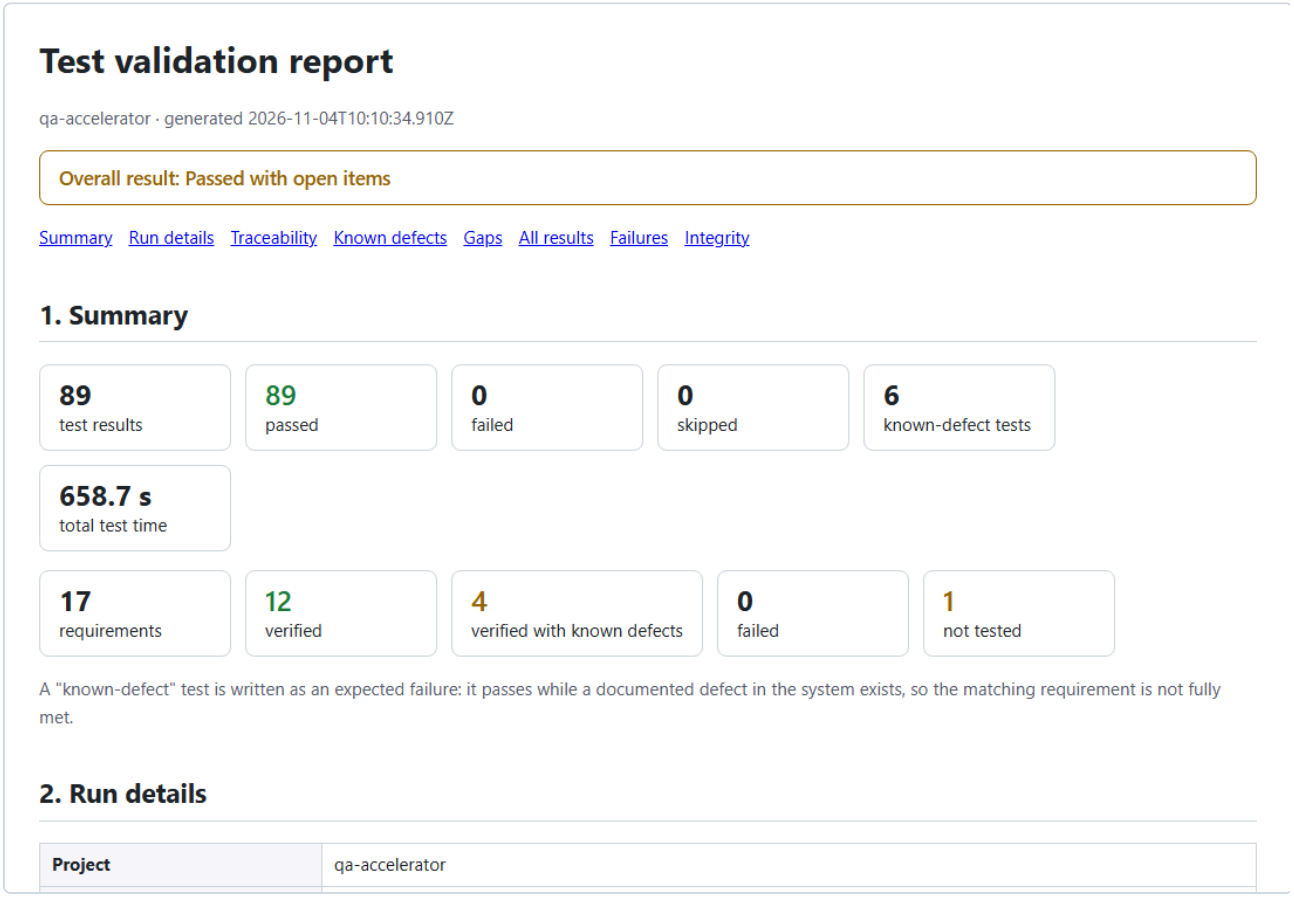
   - the matrix, with a verdict for each requirement (verified, failed, not tested, skipped);

    - the known defects, with the requirement each one affects;
    - a list of tests with no requirement;
    - a list of IDs used in tests that are missing from the requirements file. Use `node tools/traceability.mjs --strict` as a gate: it exits with an error if a requirement is failing, untested or unknown.

4. `npm run evidence` adds a manifest (commit, branch, who ran it, run URL, Node, Playwright and OS versions, lock-file hash, application image digests) and a `SHA256SUMS` file, and puts everything in `reports/regulated/`.

5. The same step writes `reports/regulated/validation-report.html`: one self-contained page with the overall result, run details, the traceability matrix, known defects, gaps, every test result and any failures. It needs no server and no other files, so it is the one document to hand to a reviewer. `npm run report:validation` also makes a PDF of it (needs Chromium installed).

6. `npm run test:capture` records a screenshot, a video and a trace for every test, and burns a live clock into each page: local time with milliseconds and UTC, plus the commit, browser and test name. Every screenshot and every video frame therefore shows when it was taken, without relying on file properties. The report's results table links the files and shows each test's start time in UTC. Setting `EVIDENCE_STAMP=true` adds the clock to any run, including the screenshots of failed tests in a normal run.



7. In CI the pack is built only when you start the workflow by hand and tick **regulated pack** (tick the capture option too for a full validation run). It is uploaded as the `evidence-<browser>` artifact. Anyone can check it was not changed:

```
cd reports/regulated && sha256sum -c SHA256SUMS
```

A requirement with no tests shows as "Not tested". That is deliberate: the sample list includes one (an order confirmation email), so the gap is visible instead of hidden.

Reading a verdict

Verdict	Meaning
Verified	At least one test, and every linked test passed.
Verified (known defect)	The requirement is not fully met: a documented defect is written as an expected failure. Raise a deviation. It is shown in amber.
Failed	A linked test failed or broke.
Not tested	No test is linked to the requirement.
Skipped	Every linked test was skipped.

Domain notes

These are the kinds of checks clients in these areas usually ask for. The demo application doesn't have them, so they are not in the repository, but the framework's layers fit them.

- **Pharmacovigilance:** E2B(R3) ICSR XML validation against the schema, MedDRA coding, expedited-reporting deadlines, duplicate-case detection, an audit-trail entry for every change to a case, and role-based access. The database layer is the natural place to check audit-trail rows.
- **Banking and insurance:** maker-checker approval flows, idempotent transactions, reconciliation between systems, rounding and currency rules, limits and fraud rules, and the PCI-DSS and security checks that apply.
- **Both:** synthetic or masked test data only, and no real personal data in test environments, logs or reports.

Documents

- dry-run-and-formal-run.md: the two-run process, the archive zip, the comparison, the change record and the CI controls.
- ../reporting.md: standard versus regulated reports, where everything is, and every way to switch screenshots and videos on.
- requirements.json: the sample requirement list read by `npm run traceability`.

- validation-plan-template.md: a starting point for the test and validation plan.
- test-summary-report-template.md: a starting point for the report after a run.
- repository-controls.md: the repository and pipeline settings to switch on.
- air-gapped-and-self-hosted.md: running without public CI or internet access.